

Week 7

Authentication and Key Distribution



THE UNIVERSITY OF
SYDNEY

Dr. Thilini Dahanayaka

School of Computer Science, The University of Sydney

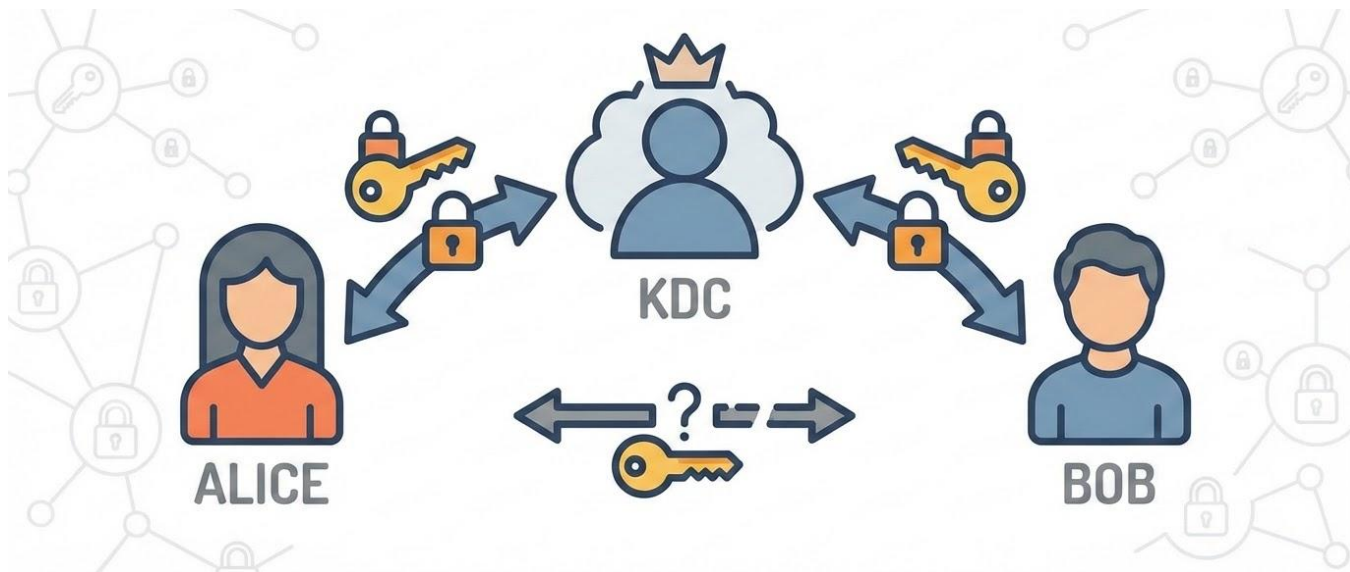
Agenda

- Symmetric key distribution (using symmetric encryption)
- Symmetric key distribution (using asymmetric encryption)
- Public-key/Asymmetric key distribution and digital certificates
- Public-key Infrastructure
- Personal identity authentication
- Authentication and key establishment (AKE)
- Remote user-authentication and Kerberos

Recommended Reading

- Cryptography and Network Security – Seventh Edition by William Stallings
 - **Chapter 14** – Key Management and Distribution
 - **Chapter 15** – User Authentication
- Security Engineering – Third Edition by Ross Anderson
 - **Chapter 4** – Protocols

1. Key Distribution



Key Distribution

- We are almost ready to understand how cryptographic protocols protect our electronic communications
 - **One last factor remains:** How do people/entities get the keys they need to communicate securely?



The Key Distribution Problem - The challenge of securely establishing shared secret keys (or authentic public keys) between parties who wish to communicate securely, without relying on a pre-existing secure channel to do so.

DH Key Exchange

Need to know other party's public key in a reliable way

Hybrid Cryptography

Need to know other party's public key in a reliable way

MACs/AES

Need a pre-shared symmetric key



No matter what primitive we use, a bootstrapping problem remains. This lecture addresses it.

Attacker Model: Dolev Yao

- We upgrade to the Dolev-Yao model, which is a stronger, more realistic adversary that reflects real-world network conditions where the attacker controls the medium entirely.
- "The attacker carries the message."
- The Dolev-Yao attacker **has full control over the network**:
 - Eavesdrop on any message in transit
 - Delay, delete, modify, replay, and inject messages
 - Impersonate any legitimate party
- ... but is **fundamentally limited by cryptography**
 - Cannot modify an integrity-protected message without the receiver noticing
 - Cannot decrypt a message encrypted with a key they don't hold



The Problem of Key Distribution: Boyd's Theorem



Boyd's Theorem - Assuming the absence of a secure channel, two entities cannot establish an authenticated session without the existence of an entity that can mediate between the two and which both parties trust and have a secure channel with.

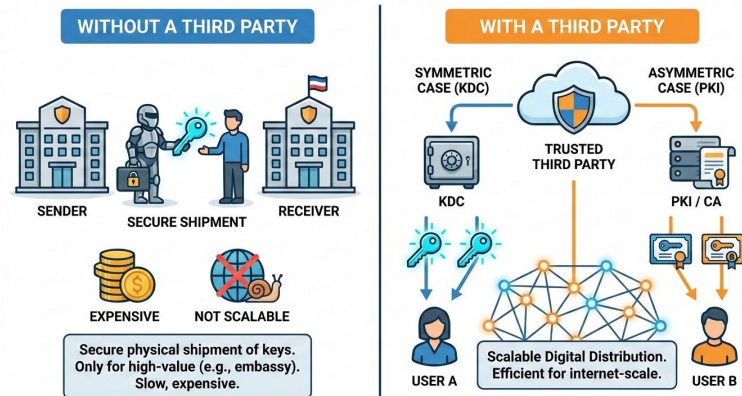
- **Rephrasing**
 - Alice and Bob cannot securely establish keys between them if they do not already have existing, established keys.
 - Only way is 'introduction' via a third party, with which both Alice and Bob have established keys already.

Diffie-Hellman Recap

- Diffie-Hellman key exchange is secure if the **attacker is passive**.
- With an **active attacker**, we need to protect the Diffie-Hellman key exchange with some form of origin authentication
 - MAC → needs a previous symmetric key exchange
 - Signature → public key needs to be known, Also correct mapping (i.e., entity/origin ↔ public key)
- We need the above protection only to protect the Diffie-Hellman exchange. Once the exchange has been completed securely once, we do have a secure channel.
 - Diffie-Hellman offers **forward secrecy** – i.e., *The session keys used to encrypt and decrypt information change frequently and automatically. This ongoing process ensures that even if the most recent key or the long-term key is breached, a minimal amount of sensitive data is exposed.*

Options for Key Establishment and Distribution

- Without a third party
 - Secure physical shipment of keys
 - Only done for high-value communication - e.g., with embassy
 - Expensive and not scalable to internet-scale
- With a third party
 - **Symmetric Case** - Key Distribution Centers (KDC)
 - **Asymmetric Case** - Public Key Infrastructures (PKI)



2. Symmetric Key Distribution (Using Symmetric Encryption)



Symmetric Key Distribution - Using a Trusted Third Party

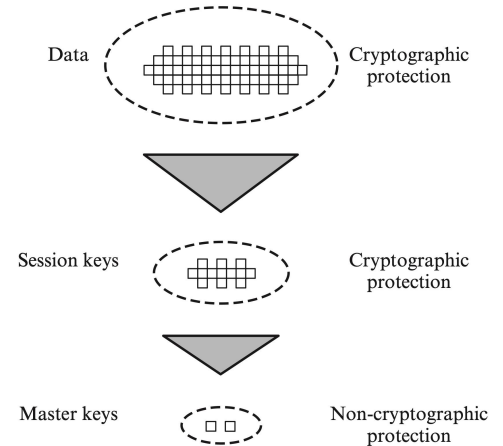
- Two parties wishing to communicate securely face a bootstrapping problem: how do they share a key without already having one?
- **The solution:** introduce a **Key Distribution Center (KDC)** - a mutually trusted third party
- Each party shares a long-term master key with the KDC (established out-of-band)
- The KDC uses these to broker **session keys** for individual communications

Key Distribution Center (KDC)

- The KDC is always online and shares a long-term key with every member
 - These keys are pre-configured by admins
 - In practice: the key is derived from a (strong) password
- Basis for the famous Kerberos protocol (Discussed Later)
 - Used across Windows, Linux, UNIX, macOS, AIX, and Solaris
 - Active Directory relies on Kerberos

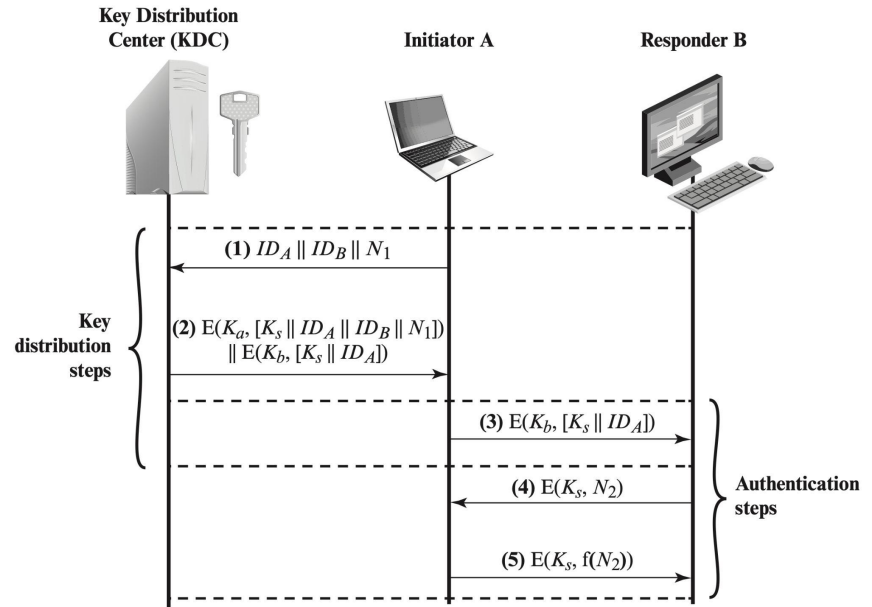
Key Hierarchy

- KDC-based distribution relies on a two-level key hierarchy.
- **Master key**: shared between the KDC and each end system or user
 - Distributed non-cryptographically — e.g., physical delivery, or bundled with managed software
 - Long-lived; used only to protect session keys
- **Session key**: used for actual communication between end systems
 - Obtained from the KDC, transmitted encrypted under the master key
 - Short-lived; one per logical connection



A Key Distribution Scenario

- Let us assume:
 - That each user shares a **unique master key** with the key distribution center (KDC)
 - User A wishes to establish a **logical connection** with B and requires a **one-time session key** to protect the data transmitted over the connection
- N_1 unique identifier, also called **nonce** (e.g., can be a timestamp, a counter, a random number) to prevent masquerade
- One-time session key K_s
- K_a, K_b master keys
- ID_A, ID_B identifier of A, B (e.g., network address)



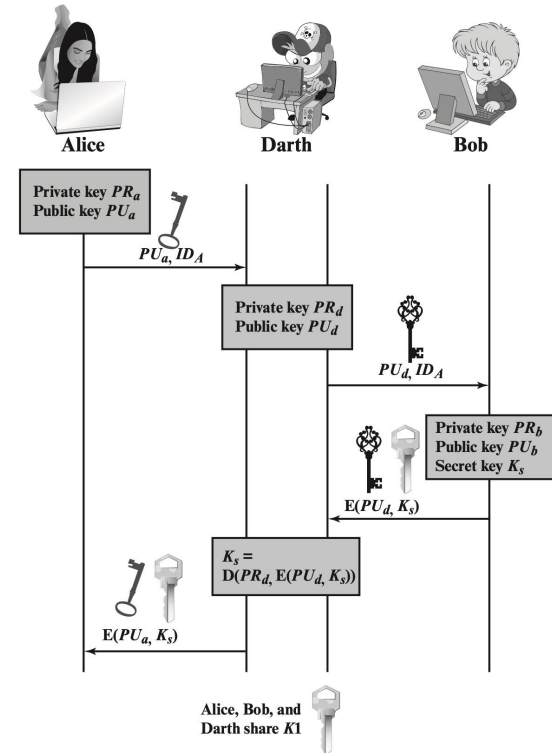
This is also called as the **Needham-Schroeder Protocol**
(We will revisit this again in detail later)

2. Symmetric Key Distribution (Using Asymmetric Encryption)



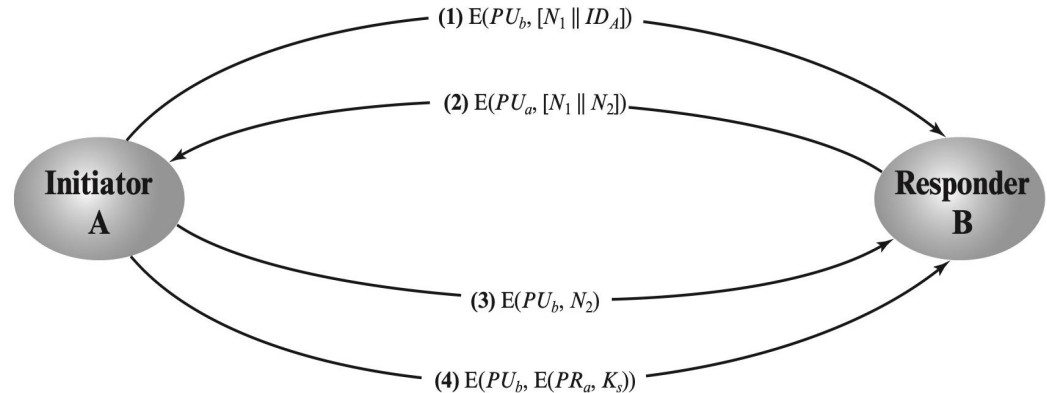
Simply Sharing the Public Key doesn't work

- We know that we can use hybrid cryptography if we reliably know the public key of the receiver.
- However, simply advertising the public key doesn't work and is vulnerable to person in the middle attacks.



Symmetric Key Distribution with Confidentiality and Authentication

- If we assume that the two parties reliably know each others public keys, we can improve upon hybrid cryptography to add authentication.
- PU_b, PU_a : A, B's public keys
- N_1, N_2 : A and B's nonce respectively
- K_s : secret key (or symmetric key)
- PR_a : A's private key
- ID_a : identifier of A



3. Public Key Distribution and Certificates



Public Key Distribution - Possibilities

1) Public announcement:

- Any participant can send his or her public key to another participant or broadcast the key
- **Weakness:** anyone can forge a public announcement

2) Publicly available directory:

- Maintaining a publicly available dynamic directory of public keys
- More secure than an individual public announcement
- **Weakness:** if the private key of the directory authority is compromised, the adversary could eavesdrop on messages sent to any participant

3) Public-key authority:

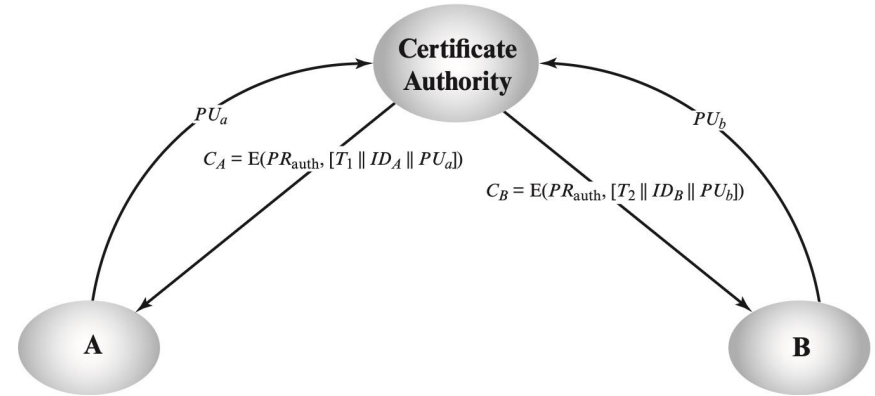
- Tighter the control over the distribution of public keys from the directory
- Based on Needham-Schroeder protocol
- **Weakness:** could make public-key authority be a bottleneck in the system

4) Public-key certificate:

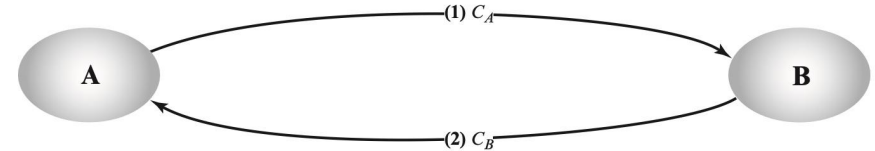
- Can be used by any participant to exchange keys without contacting a public-key authority
- **Better option and what we use today!**
- Central idea: **certificates** - used in Public Key Infrastructures (PKIs)

Certificate Creation

- Public-key certificate: the certificate consists of
 - A public key - PU
 - An identifier of the key owner - ID
 - Whole block signed by a trusted third party – Certificate Authority (CA)



(a) Obtaining certificates from CA



(b) Exchanging certificates

PR_{auth} : authority's private key

T : timestamp

X.509 Certificate

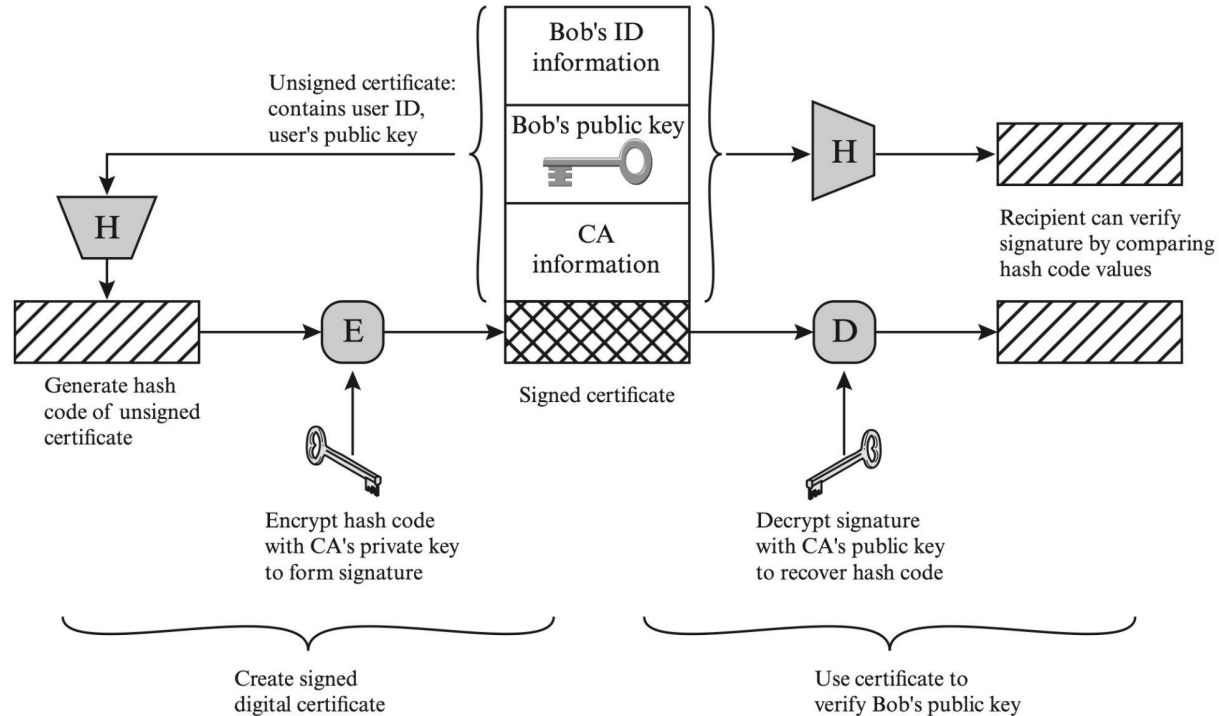


Digital Certificate - A digital certificate is a cryptographic binding between an identifier and a public key that is to be associated with that identifier. An issuer creates the binding and takes the role of a trusted third party (TTP), in accordance with Boyd's theorem.

X.509

- X.509 standard defines the format of public-key certificates
- X.509 is based on the use of public-key cryptography and digital signatures
- Used in many modern cryptographic protocols on the Internet

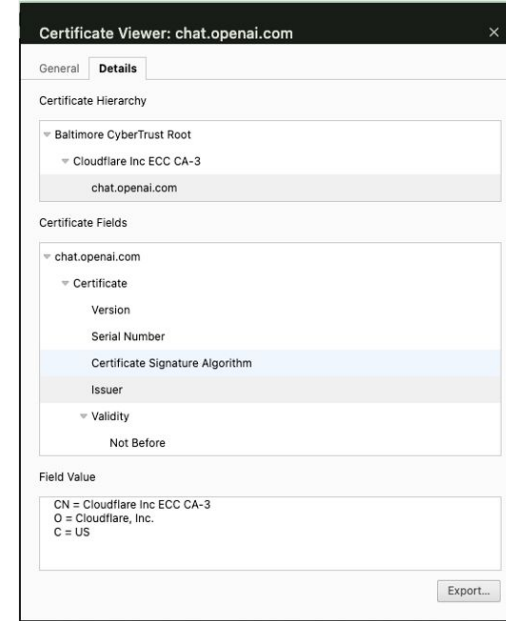
X.509 Scheme



X.509 Certificate (Version 3)

X509v3 Certificate		
Version	Serial no.	Sig. algo.
Issuer		
Validity	Not Before	Not After
Subject		
Subject Public Key Info		
	Algorithm	Public Key
X509 v3 Extensions		
	CA Flag, EV, CRL, etc.	
Signature		

X.509 Format - Example



Example certificate taken from Google Chrome for ChatGPT

4. Public Key Infrastructure (PKI)



Public Key Infrastructure (PKI)

- **The Problem:** How does Bob know a given public is actually Alice's public key?
- **The Solution:** Public Key Infrastructure (PKI).
- **The "Trust Anchor":** This is achieved through Certificates issued by a trusted third party (the [Certificate Authority](#)).



Public Key Infrastructure (PKI) is a system of hardware, software, people, policies, and procedures needed to create, manage, distribute, use, store, verify, and revoke digital certificates and manage public-key encryption.

PKI - The Chain of Trust and Root CAs

- **The Chain of Trust:** Trust is transitive.

- If I_{Root} signs I_{Int} , and I_{Int} signs User X:
- I_{Root} vouch I_{Int} vouch User X

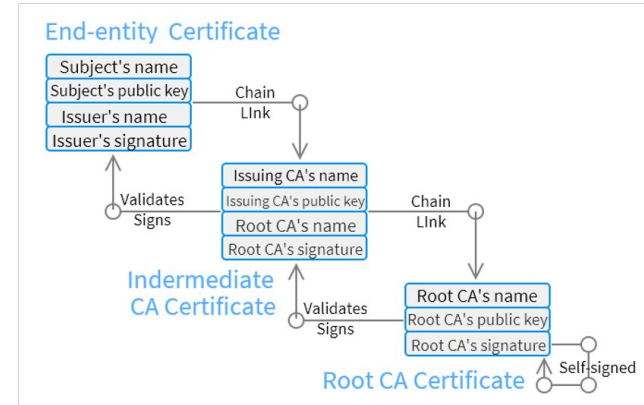
$$I_{Root} \xrightarrow{\text{vouch}} I_{Int} \xrightarrow{\text{vouch}} \text{User } X$$

- To verify X, you follow the arrows back to a source you already trust.

- **The "Self-Signed" Root:**

- The **Root CA** sits at the top. It signs its own certificate.
- **Trust Gap:** You cannot verify a Root CA through a chain.
- **The Solution:** Root certificates are **pre-installed** in your browser or OS.

- **Verification:** To trust Alice, Bob's computer checks every signature in the chain until it hits a pre-installed Root.



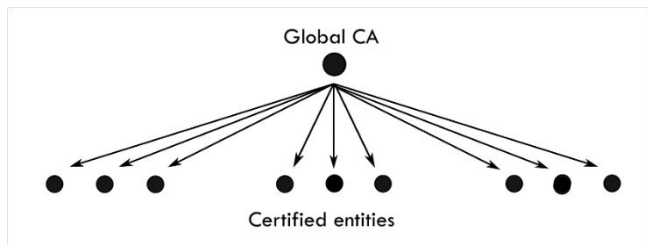
CA - Certificate Authority

PKI - Identity Verification when Issuing Certificates

- The issuance process embeds a critical step
- Before issuing a certificate, the issuer must be sure that they
 - Are issuing to the correct identity **and**
 - The identity really holds that particular key
- Easier said than done: which verification process is acceptable to everyone?
- There are industry-agreed steps that must be carried out
- Vary depending on purpose of certificate, and its value
- Baseline verification:
 - Commonly used for domains
 - Proof of ownership: CA requests domain operator to place some file on the web server, or be able to receive email under that domain name
- Extended verification:
 - High-assurance certificates that require legal documentation and background checks.
 - More expensive and rigorous, providing a higher level of "human" trust.

Hierarchical PKI

Naive form: one global issuer trusted by everyone



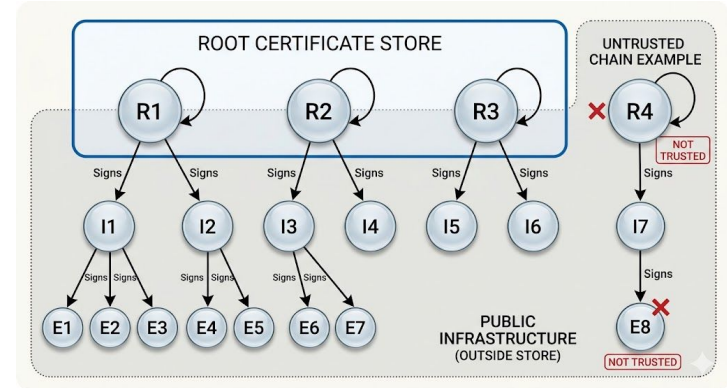
- Not very practical
- Identity verification in different jurisdictions can be different
- Too much centralization

Solution: Many global CAs

- Allow many CAs - accept them all as equals
- But how do you get users to 'trust' them and be able to use them?
- Solution: **the public keys of 'trusted' CAs** are shipped with your **OS and/or browser**
 - Users do not actually get to choose which CAs they trust
 - The nature of the verification process is defined between vendors and CAs

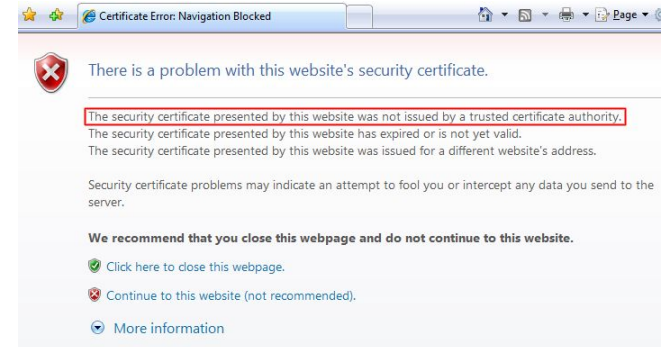
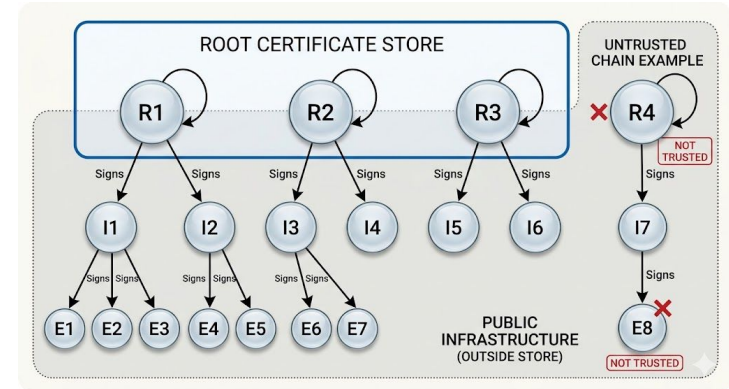
Root Certificate Store

- **The Repository:** Root stores hold the certificates of trusted Certificate Authorities (CAs).
- **Trust Anchors:** These stores specifically contain the **self-signed Root Certificates** that serve as the mathematical "starting point" for all validation chains.
- **Application Use:** Every application that uses X.509 (e.g., browsers) or the Operating System itself must maintain a root store.
- **Recall:** Root certificates are self-signed because they are the apex of the hierarchy; they cannot be verified mathematically via a higher authority and must be trusted by decree through pre-installation.



Root Certificate Store

- **The Repository:** Root stores hold the certificates of trusted Certificate Authorities (CAs).
- **Trust Anchors:** These stores specifically contain the **self-signed Root Certificates** that serve as the mathematical "starting point" for all validation chains.
- **Application Use:** Every application that uses X.509 (e.g., browsers) or the Operating System itself must maintain a root store.
- **Recall:** Root certificates are self-signed because they are the apex of the hierarchy; they cannot be verified mathematically via a higher authority and must be trusted by decree through pre-installation.



Why Intermediate Certificates?



Intermediate Certificates are "middle-link" certificates that are neither the trusted Root nor the final End-Entity. They form a bridge in the Chain of Trust.

- **Delegated Authority (Sub-CAs):**

- Allows a Root CA to delegate signing powers to different organizations, departments, or geographical regions.
- Enables a hierarchical "web of trust" without requiring every user to contact the main Root CA directly.

- **Risk Mitigation & Root Protection:**

- The Root CA's private key can be kept offline in a highly secure, air-gapped location (often a physical safe).
- Daily signing operations (issuing certificates to websites/users) are handled by the Intermediate CA.
- If an Intermediate CA is compromised, it can be revoked without needing to replace the Root CA in every user's Root Store, a process that would take years.

The PKI X.509 (PKIX) Model

- While X.509 defines the "data structure" of a certificate, the PKIX model defines how that data moves across the global internet.
- **The IETF PKIX Profile:** The Internet Engineering Task Force (IETF) developed the PKIX profile (RFC 5280) to ensure different vendors (e.g., Microsoft, Google, Cisco) can all interoperate using the same rules.
- **Standardizing the Lifecycle:** PKIX doesn't just define the "ID card"; it defines the entire ecosystem:
 - **Certificate Revocation Lists (CRLs):** The "blacklist" of certificates that were compromised before their expiry date.
 - **Online Certificate Status Protocol (OCSP):** A real-time way for Bob to ask the CA, "Is Alice's key still valid right now?"
- **Trust Models:** PKIX formalizes the Hierarchical PKI structure you just showed in the previous diagram, establishing the technical "rules of the road" for Certificate Authorities.

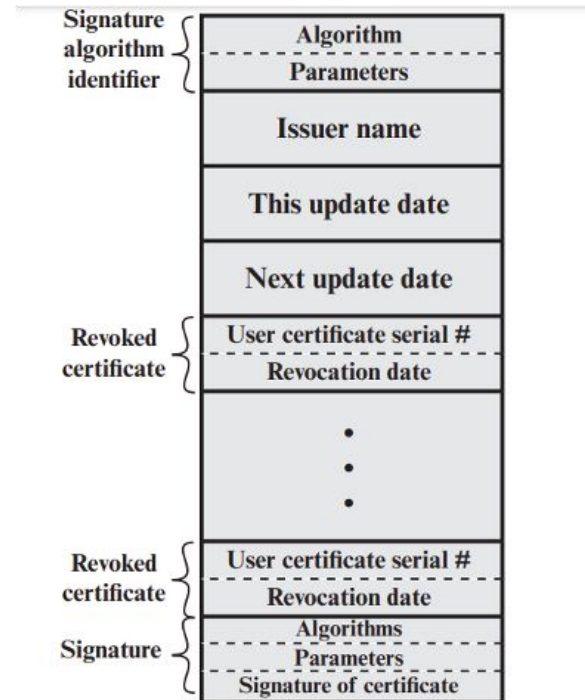
Certificate Revocation

- A certificate is merely a signed statement that a binding was valid at the moment of issuance.
- We must invalidate a certificate if the "Trust Foundation" collapses before the expiration date:
 - **Private Key Compromise:** The key is stolen, leaked, or suspected of being accessed by an unauthorized party (e.g., a server breach).
 - **Identity Fraud/Change:** It is discovered that the identity verification was flawed, the domain ownership changed, or the entity (company/employee) no longer exists.
 - **CA Compromise:** The Certificate Authority that issued the cert is itself compromised, making all its "promises" untrustworthy.
- To be sure the binding is valid at the time of interest (the moment the receiver receives the message), they must be able to perform a real-time revocation check.

Certificate Revocation Lists (CRLs)

CRL: list of certificates that are considered revoked

- Should be issued, updated and maintained by every CA
 - Certificates are identified by serial number
 - Every CRL must be time stamped and signed
- Technically, a browser (client) should download CRL (and update it after the given time), and lookup a host certificate every time it connects to a server
- Too slow, not done (anymore) in practice



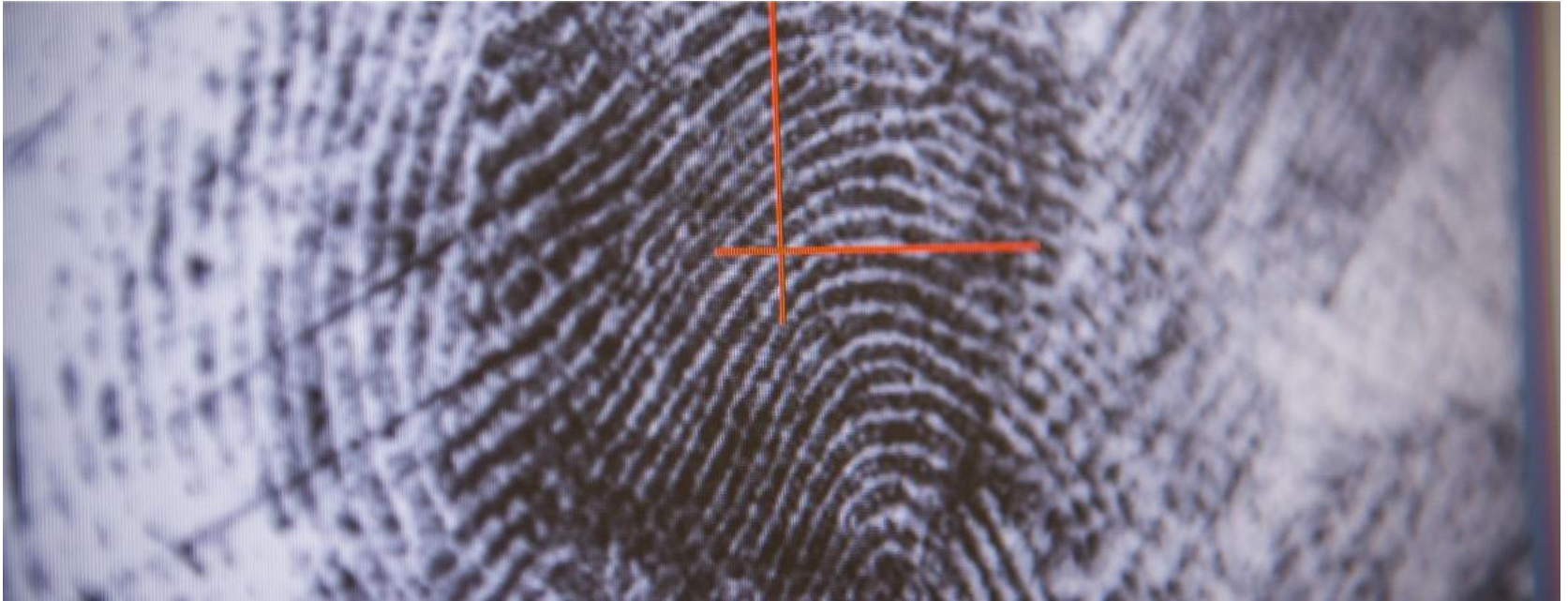
Online Certificate Status Protocol (OCSP) and OCSP Stapling

- Because CRLs can be massive and stale, we use Online Certificate Status Protocol (OCSP) for live, per-certificate status checks.
- **Standard OCSP (Client-Side Check):**
 - Mechanism: The client (Bob) queries the CA's "OCSP Responder" with a certificate serial number.
 - Response: The CA returns a signed status (Good, Revoked, or Unknown).
 - Drawbacks: Slows down the initial connection (latency) and leaks Bob's browsing history to the CA (privacy).
- **OCSP Stapling (Server-Side Solution):**
 - Mechanism: The server (Alice) fetches the signed OCSP response from the CA periodically and "staples" it to the certificate during the handshake.
 - The Staple: Bob receives both the certificate and the proof of validity in one go.
 - Benefits: Zero extra latency for Bob and total privacy from the CA.

Problems of X.509

- In modern browsers, all pre-installed Root CAs are equally trusted. A compromise of a single, obscure CA in a distant jurisdiction can be used to issue a fake certificate for any global domain.
- Verification Failures (Identity Fraud):
 - **The "Trickster" Attack:** Adversaries have successfully convinced CAs of forged identities, obtaining valid certificates for domains they do not own (e.g., the Comodo and DigiNotar incidents).
 - **Hacked CAs:** Direct attacks on CA infrastructure have allowed hackers to issue fraudulent certificates at will.
- Systemic Weaknesses Revealed by Research:
 - **Lax Certification Practices:** Many CAs prioritize speed and profit over rigorous identity verification.
 - **Operator Mismanagement:** Many website administrators deploy certificates with errors, such as mismatched domain names or expired chains.
 - **High-Value Bias:** Effective X.509 deployment is often only seen in top-tier, high-value websites; the "long tail" of the internet remains poorly secured.

5. Authentication



Authentication

- We briefly mentioned authentication in the context of access control, when we discussed things like users and privileges
 - Before you can use the filesystem, you need to login
 - The login is a form of authentication
 - After authentication, you have authorisation for certain things
- We skirted around the question, however, what **authentication actually is**

Authentication: Definition by Menezes et al.



Authentication is the process whereby one party is assured (through the acquisition of corroborative evidence) of the identity of a second party involved in a protocol, and that the second has actually participated.

- Note the elements needed to ascertain the identity
 - Corroborative evidence
 - Process between (at least) two parties
 - Involvement and participation of the second party

Authentication: Definition by Menezes et al.



Authentication is the process whereby one party is assured (through the acquisition of corroborative evidence) of the identity of a second party involved in a protocol, and that the second has actually participated.

- Note the elements needed to ascertain the identity
 - **Corroborative evidence**
 - Process between (at least) two parties
 - Involvement and participation of the second party

Corroborative Evidence

We are looking for factors that are unique to an entity. Classic categorization:

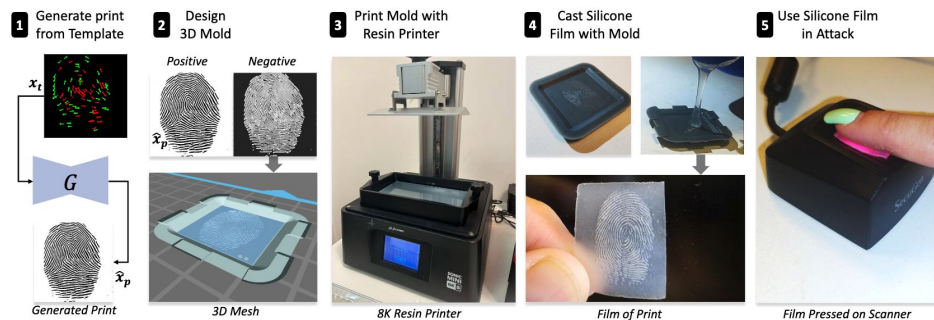
- **Possession:** Something the entity/user has (what you have)
 - Physical key, phone to send SMS messages to
- **Inherence:** Something the user is (what you are)
 - Biometrics: fingerprints, iris scan, face and voice recognition
- **Knowledge:** Something the user knows (what you know)
 - Passwords, security questions (mother's maiden name, etc.) These can be used stand-alone or in combination.

Common Problems: With Possession

- Obvious problem: **loss of the physical device**
- Others are more subtle:
 - How is the phone identified? By phone number? SIM?
 - SMS messages are sent to a number - can be rerouted
 - SS7 is the global signaling network that also does SMS - bunch of known vulnerabilities

Common Problems: With Inherence

- Biometrics: in general, **cannot change these**
- Some can be easily inferred
 - Example: fingerprints are everywhere, and even standard cameras can capture them
 - <https://www.ccc.de/en/updates/2014/ursel>
- More susceptible to forgery via recordings - Generative AI, 3D Printing



Hacmon, Y., Gorelik, K., & Mirsky, Y. (2025, August). The Threat of Deepfake Fingerprints. In *Proceedings of the 4th Workshop on Security Implications of Deepfakes and Cheapfakes* (pp. 1-8).

Common Problems: With Knowledge

- We have already discussed the usability and strength of passwords
 - Note that databases of cracked passwords exist
 - <https://haveibeenpwned.com>
- Security questions are often very standard, with predictable answers and limited possibilities
 - Mother's maiden name? - depending on culture, try Smith, Chang, Kim, Schmidt, ...
 - First car? – try Golf, Yaris, Corolla, ...
 - Social networks help collect additional information about a person

MFA - Multi-Factor Authentication

- Increased security by requiring multiple factors for authentication
 - In general: **from different categories**
 - Special form: two-factor authentication (2FA)
- Example: password and code sent as text to phone
 - Needs very motivated attacker, and assumes targeted attack, not broad sweep
- Some factors are stronger than others
 - Choose factors according to the problem and setting
 - E.g. possession is strong, but loss/forgetfulness is a problem

MFA - Multi-Factor Authentication

- 2FA is standard today for high-value sites
 - E.g. webmail providers, social networks, ...
- Often, MFA comes with mechanisms that check the plausibility
 - E.g. if a request to login is coming from an unusual country, from which the user never logged in before
 - Or if many failed login attempts occur
 - Or if IP is from a range that is known to be the origin of other attacks
 - Then, MFA is switched on or additional factors are required
 - E.g. password plus phone code plus email with access token is sent
- Implementation must follow the principle of psychological acceptability

Limits of Multi-Factor Authentication

- MFA places a bigger onus on the user:
 - Remember **psychological acceptability**!
 - Generally, MFA is more readily accepted in high-value, high-risk scenarios
- MFA with factors from the same category is not guaranteed stronger
 - E.g., asking for two passwords (which may both be weak)
- Social engineering can still (easily) overcome multiple factors!
 - Phone call that tricks the user into disclosing both password and code from phone

One-Time Passwords (OTP)

Strengthening with one-time passwords

- One-Time Passwords are valid for one login only
- Avoids the attacker who's able to record a password and tries to reuse it
- **Two categories:**
 - Static - (indexed) list of OTPs is given to user
 - Dynamic - following a challenge-response principle
 - In the latter case, the OTP will depend on the challenge
- Examples: Google Authenticator, Microsoft OTP, Yubikey, ...
- **Note:** OTP can be done as MFA
 - E.g. OTP is computed by a device the user must possess
 - E.g. challenge sent by website in form of code, type it into device to derive OTP
 - Allows to create context-specific OTPs

Authentication: Definition by Menezes et al.



Authentication is the process whereby one party is assured (through the acquisition of corroborative evidence) of the identity of a second party involved in a protocol, and that the second has actually participated.

- Note the elements needed to ascertain the identity
 - Corroborative evidence
 - **Process between (at least) two parties**
 - **Involvement and participation of the second party**

Console Login vs. Network Login

Console Login

- **Principle:** Participation guaranteed by physical keyboard input
 - Password must be typed locally by someone present
 - Even USB attack devices face rate-limiting delays

Network Login - The Participation Problem

- **Challenge:** No physical presence → difficult to prove active participation
- **Key Issues:**
 - Credentials transmitted over network can be captured
 - Replay attacks: intercepted credentials reused later
 - Cannot distinguish:
 - Legitimate user actively logging in
 - Malware sending stored credentials
 - Attacker replaying captured data

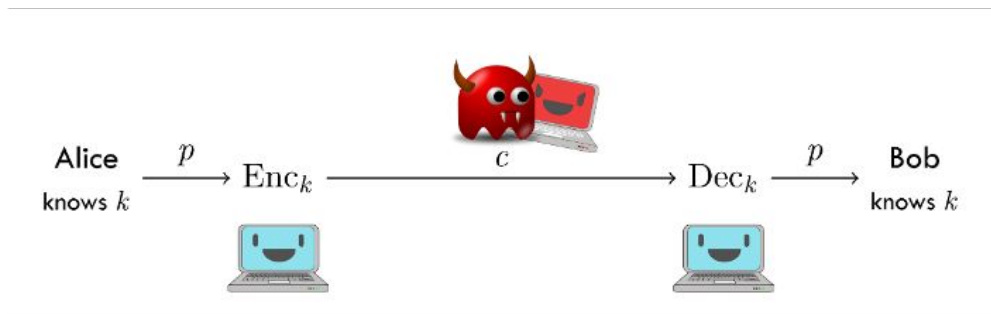


Implication: Network authentication requires additional mechanisms (challenge-response, nonces, time-based tokens) to cryptographically prove fresh, active participation beyond just "knowing" the password.

Authentication Protocols

- So far, we applied cryptography mostly to single messages
- For authentication over a network, we must do better: sequence of steps
- Sequence of well-defined steps to achieve a goal: this is a **protocol**
- If we add cryptographic protection: **cryptographic protocol**
- Warning: Composition of cryptographic operations into secure protocols is **hard**
 - Reason: even if crypto is perfect, we can leave loopholes in the protocol!
 - Just like the replay of an encrypted password

Person-in-the-Middle Attack



- Alice tries to send password p to Bob, encrypted with k
- Attacker can just record $c = \text{Enc}_k(p)$ - and replay it later!
- Bob does not actually know if Alice has participated!
- Neither MACs nor signatures help! They can be replayed as well.
- We must somehow bind the transmission of the password to the **current communication**

Building Protocols

We construct cryptographic protocols from primitives:

- Cryptographically secure random numbers
 - Essential ingredient in all that follows
- Cryptographic hash functions give us:
 - MACs (by mixing in shared secret)
- Public-key cryptography gives us:
 - Key exchange over certain untrusted channels
 - Confidentiality (→ **hybrid: encrypt symmetric keys**)
 - Origin authentication and data integrity
- Symmetric-key cryptography gives us:
 - Confidentiality (→ **actual data transfer**)
 - Origin authentication and data integrity → MACs

6. Authentication and Key Establishment



Constructing an Authentication Protocol

Setup: Alice (A) wants to authenticate to Bob (B)

Goal: A must prove possession or knowledge of something only A has

Assumptions: Alice and Bob share a secret key $k_{A,B}$ and Bob has a way to verify Alice's password

Naive Attempt #1: Send the encrypted password

Protocol: $A \rightarrow B: c = \text{Enc}_k(p)$

Problem: Replay attack

- Attacker C intercepts c
- $C \rightarrow B: c$ (replays the same ciphertext)
- Bob accepts C as Alice!
- C doesn't need to know p or k , just replays c

Naive Attempt #2: Add Message Authentication Code (MAC)

Protocol: $A \rightarrow B: c, \text{MAC}_{k_{A,B}}(c)$

Why still not secure:

- MAC prevents tampering but NOT replay
- Attacker C can still capture and replay both c and $\text{MAC}_{k_{A,B}}(c)$
- $C \rightarrow B: c, \text{MAC}_{k_{A,B}}(c)$
- Bob verifies MAC successfully and accepts C as Alice!



Key insight: We need a way to ensure freshness and prove active participation, not just knowledge of the key.

Challenge-Response with Nonces

- In the following, let $\{m\}$ denote an encrypted and integrity-protected message. A **nonce** is a **'random number used once'** – also mentioned in previous slides.
- We play:
 - $A \rightarrow B : N_A$
 - $B \rightarrow A : \{N_A, N_B\}$
 - $A \rightarrow B : \{N_A, N_B + 1\}$
- B has assurance now that A has participated and reacted to precisely the second message
- Because she was able to read N_B (the challenge) and increment it (response)
- **Note:** nonces can be replaced with timestamps in some cases, but we do not show this here.

AKE: Authentication and Key Establishment

Most protocols **combine authentication and key establishment**. There are multiple approaches.

- Password-based methods (with challenge-response to prevent replay)
 - Use nonces/timestamps to ensure freshness
 - Derive session keys from the password exchange
 - Examples: encrypted key exchange protocols
- Diffie-Hellman based methods
 - Establish shared secret without pre-shared password
 - Combine with authentication (certificates, signatures)
 - Examples: TLS, SSH key exchange
- Important rules for session key management:
 - Always generate 'session keys' from your actual, master keys
 - Use session keys for one communication 'session', then delete them
 - Never reuse session keys
 - Both parties should be involved in the key creation.

Simple AKE

Idea: use long-term asymmetric keys to protect short-term symmetric keys.

- Simplest hybrid crypto: Alice generates symmetric key k .

$A \rightarrow B : Enc_{k_{e,B}}(k), Sig_A(k)$ (use k for further encryption)

- Better schema for hybrid crypto:

$A \rightarrow B : Enc_{k_{e,B}}(k_A), Sig_A(k_A)$

$B \rightarrow A : Enc_{k_{e,A}}(k_B), Sig_B(k_B)$

- Then use $k = f(k_A \parallel k_B)$ for encryption.
- Now both sides are involved in key creation. Even if one chooses a poor key half by accident, the scheme remains secure.
- Variants of this are in use (but are being phased out)

Diffie-Hellman Key Exchange - Recap

Let p be prime, and g , a generator for $\mathbb{Z}_p^*$

Alice

- Choose random value $a < p$
- Compute $X = g^a \bmod p$
- Send X to Bob
- Compute $k = Y^a \bmod p$

Bob

- Choose random value $b < p$
- Compute $Y = g^b \bmod p$
- Send Y to Alice
- Compute $k = X^b \bmod p$

➡ $Y^a \bmod p = (g^b)^a = g^{ab} = (g^a)^b = X^b \bmod p$

➡ Both sides obtain the same k value

DH Key exchange allows key establishment over an insecure channel, under a passive attacker.

AKE with Diffie-Hellman Key Exchange

Let p be prime, and g , a generator for $\mathbb{Z}_p^*$

Alice

- Choose random value $a < p$
- Compute $X = g^a \bmod p$
- Send X , $\text{Sig}_A(X)$ to Bob
- Compute $k = Y^a \bmod p$

Bob

- Choose random value $b < p$
- Compute $Y = g^b \bmod p$
- Send Y , $\text{Sig}_B(Y)$ to Alice
- Compute $k = X^b \bmod p$

- In practice, we **sign Diffie-Hellman values**
- This protects against an active attacker trying to modify them
- Alice and Bob use their (long-term secret) private keys for that, e.g. RSA keys

Forward Secrecy

- Using Diffie-Hellman every time we run a protocol gives us fresh session keys every time ('ephemeral' keys).
- **But must choose fresh a, b every time and delete old ones.**
- This is **Forward Secrecy**:
 - If the attacker learns long-term signing keys: all messages protected with session keys remain secure.
 - Attacker learns one pair (a_i, b_i) ? All messages protected with (a_j, b_j) , $i \neq j$, are still secure.



Forward Secrecy (often called **Perfect Forward Secrecy** or **PFS**) is a property of key-agreement protocols that ensures a session key will not be compromised even if the long-term private keys (the "master keys") are compromised in the future.

Modern Cryptographic APIs

- The Problem: Too many ways to make mistakes
 - Common developer errors:
 - Poor key lengths or weak random values (predictable seeds)
 - Wrong or no padding (leading to padding oracle attacks)
 - Insecure block cipher modes (ECB mode reveals patterns)
 - Mixing encryption and authentication incorrectly
 - Reusing nonces/IVs with stream ciphers
- Modern API Design Philosophy
 - Don't give developers dangerous choices - make safe defaults the only option
 - Real-world examples:
 - libsodium / NaCl
 - AWS Encryption SDK, Google Tink
 - TLS 1.3 → Removed weak cipher suites entirely (no RSA key exchange, no CBC mode)

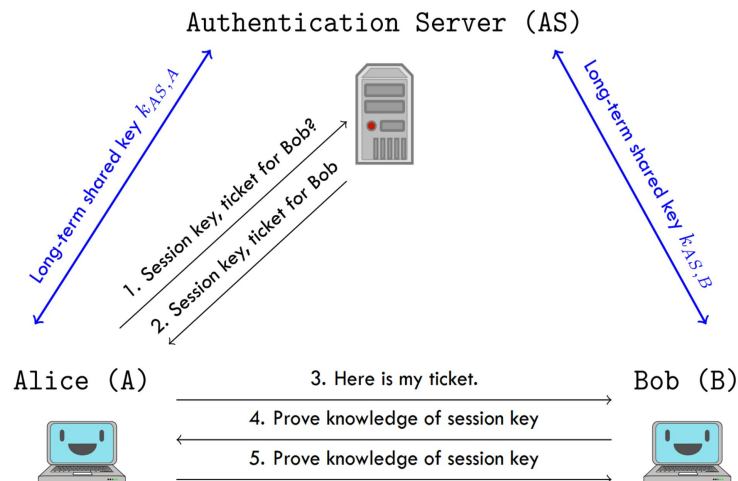
7. Kerberos



Needham-Schroeder Protocol: Basic Idea

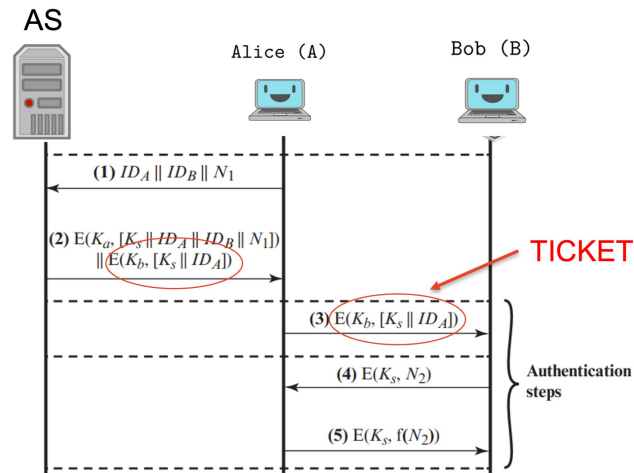
Needham-Schroeder protocol

- **Idea:** intermediary 'Authentication Server' that helps Alice and Bob
- Protocols help establish a session key between two users over an insecure network
- Several variants exist
- We focus on the one that uses symmetric keys
- Needham-Schroeder protocol is a way to build a **Key Distribution Centre (KDC)**



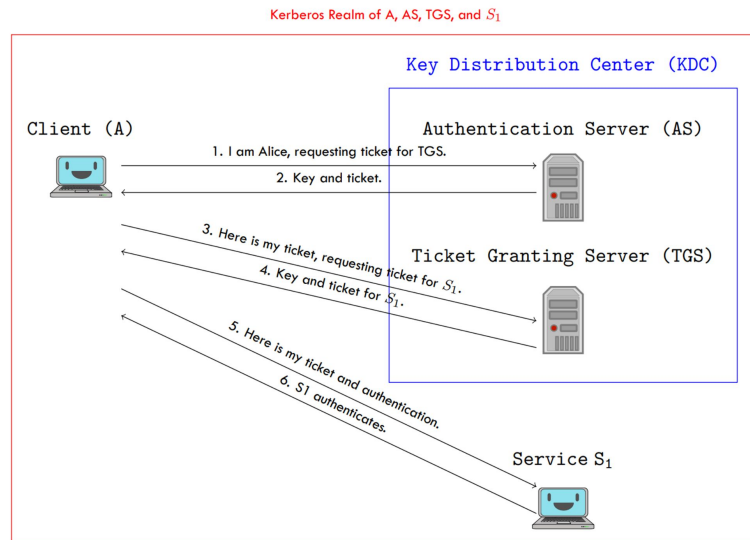
Needham-Schroeder Protocol

- Initiate; nonce identifies the session
 - $A \rightarrow AS : ID_A, ID_B, N_1$
- AS creates session key K_s ; encrypts with shared with Bob ('ticket')
 - $AS \rightarrow A : E(K_a, [K_s \| ID_A \| ID_B \| N_1]) \parallel E(K_b, [K_s \| ID_A])$
- Forward the ticket
 - $A \rightarrow B : E(K_b, [K_s \| ID_A])$
- Prove knowledge of session key (freshness!)
 - $B \rightarrow A : E(K_s, N_2)$
- Prove knowledge of session key and nonce, f is a generic function that modifies the value of nonce
 - $A \rightarrow B : E(K_s, f(N_2))$

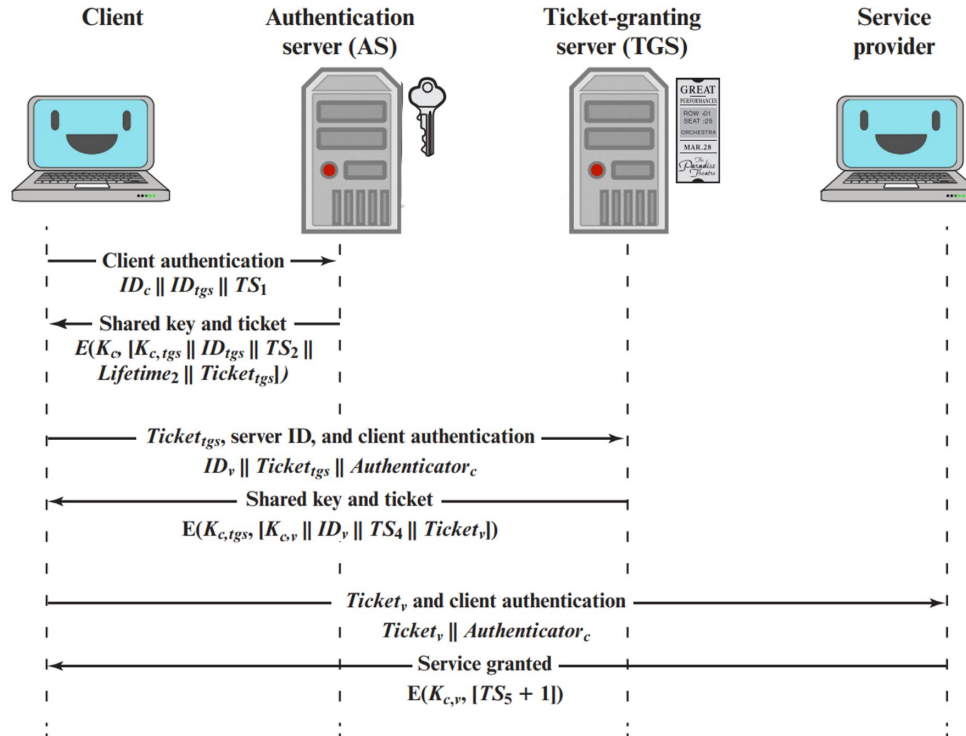


Kerberos

- Wide-spread protocol for authentication and access control
- Much deployment as a backend protocol
- **Built on Needham-Schroeder**, it has remarkable features:
- Ticket concept to allow users to access services
- Users authenticate with a password and request access to a server
- Timestamps give tickets a lifetime, after which reauthentication is necessary
- Can be used to 'federate' several administrative domains



Kerberos



Kerberos

1. Client requests ticket-granting ticket:

$$C \rightarrow AS : ID_c || ID_{tgs} || TS_1$$

- ID_c : client identity
- ID_{tgs} : requests access to TGS
- TS_1 : allow AS to verify that the client's clock is synchronized with that of AS

2. AS returns ticket-granting ticket:

$$AS \rightarrow C : E(K_c, [K_{c,tgs} || ID_{tgs} || TS_2 || Lifetime_2 || Ticket_{tgs}])$$

- K_c : encryption is based on client's key and protecting content of message
- $K_{c,tgs}$: session key for client and TGS
- $Ticket_{tgs}$: for client to access TGS
- TS_2 : time this ticket was issued, $Lifetime_2$: the lifetime of this ticket
- ID_{tgs} : confirm ticket is for TGS

Kerberos

3. Client requests service-granting ticket:

$$C \rightarrow TGS : ID_v || Ticket_{tgs} || Authenticator_c$$

- $Authenticator_c$: Generated by client to validate ticket
- ID_v : client requests access to server V

4. TGS returns service-granting ticket:

$$TGS \rightarrow C : E(K_{c,tgs}, [K_{c,v} || ID_v || TS_4 || Ticket_v])$$

- $K_{c,tgs}$: key shared only by client and TGS to protect content of message
- $K_{c,v}$: session key for client and server V
- ID_v : confirm ticket is for server V
- $Ticket_v$: ticket to access server V

Kerberos

5. Client requests service:

$$C \rightarrow S_v : Ticket_v || Authenticator_c$$

- $Ticket_v$: Assures server that this user has been authenticated by AS

6. Optional authentication of server to client, service granted:

$$S_v \rightarrow C : E(K_{c,v}, [TS_5 + 1])$$

- $K_{c,v}$: assures client that this message is from V
- $TS_5 + 1$: Assures C that this is not a replay of an old reply

Recap

- **We discussed:**

- Symmetric key distribution (using symmetric encryption)
- Symmetric key distribution (using asymmetric encryption)
- Public-key/Asymmetric key distribution and digital certificates
- Public-key Infrastructure
- Personal identity authentication
- Authentication and key establishment (AKE)
- Remote user-authentication and Kerberos

